DESIGN PROJECT

LLM Hallucination Detection using EigenScore

PRESENTED BY

Aayaan Ali Khan (202411001) | **Shlok Bhavsar** (202411018) | **Prapti Chotaliya** (202411030)

Indian Institute of Information Technology Vadodara

1. Problem Statement

Large Language Models often **hallucinate**: they give confident answers that are wrong, in the same tone as correct ones.

> **Goal:** for every answer, output a **hallucination score** that says how likely it is to be wrong, **without knowing the correct answer**, on an **8 GB laptop GPU**.

2. The Idea: Ask Again, Check Agreement

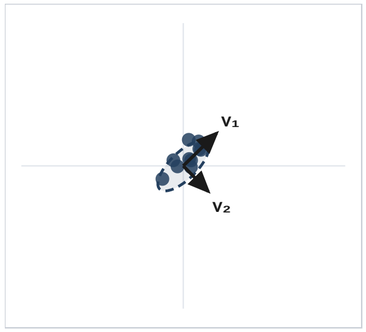
Q: "In which year did the bridge open?", asked 5 times

Model knows	1932	1932	1932	1932	1932	✓ trust
Model guesses	1932	1928	1936	1941	1930	⚠ flag

Fig. 1. Answers that agree signal knowledge; answers that scatter signal guessing (illustrative).

We compare the answers not by their words but by the model's **internal representation** of each answer: a list of numbers read from inside the network.

3. Intuition: Spread = Eigenvalues



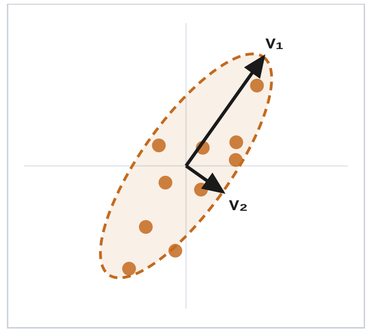
Answers agree

small cloud → small eigenvalues

Eigenvalues (spread along v_1, v_2)

λ1 = 0.04

λ2 = 0.01



Answers disagree

large cloud → large eigenvalues

Eigenvalues (spread along v_1, v_2)

λ1 = 0.66

λ2 = 0.07

Fig. 2. Illustrative 2-D view: each dot is one answer's internal representation. Arrows v_1, v_2 are eigenvectors (main directions of spread); bar lengths are eigenvalues (amount of spread).

$$\text{EigenScore} = \text{average of } \log_{10}(\text{eigenvalues})$$

Answers agree → small eigenvalues → **low score** · disagree → large eigenvalues → **high score**

4. Why Internal States, Not Words

Answer 1: **"1932"**

Answer 2: **"in nineteen thirty-two"**

Same meaning, different words.

Word overlap: 0%
→ wrongly looks like disagreement

Internal vectors: close together
→ treated as agreement (illustrative)

5. Illustrative Example (K = 3, d = 4)

Case A: answers agree

```
Z = [1.0 0.2 0.6 0.4] ← answer 1
    [1.1 0.2 0.6 0.4] ← answer 2
    [1.0 0.3 0.6 0.4] ← answer 3
```

C = cov(Z), 3×3 matrix
eigenvalues of C + 0.001·I:
0.362, 0.0017, 0.0011
score = mean($\log_{10}$) = **−2.05 → trust**

Case B: answers disagree

```
Z = [1.0 0.2 0.6 0.4] ← answer 1
    [0.1 0.9 0.3 0.7] ← answer 2
    [0.5 0.4 1.0 0.0] ← answer 3
```

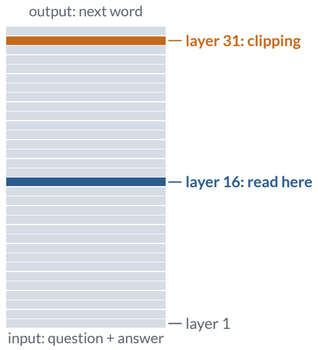
C = cov(Z), 3×3 matrix
eigenvalues of C + 0.001·I:
0.306, 0.114, 0.0022
score = mean($\log_{10}$) = **−1.37 → flag**

When responses agree, one eigenvalue dominates and the rest collapse to ≈ 0.001, giving a low score. The computation is identical to the full-scale system.

6. Illustrative vs. Full-Scale Experiment

Quantity	Illustrative	Full-scale experiment
Answers per question (K)	3	**10**
Embedding dimension (d)	4	**4,096**
Where numbers come from	synthetic values	**layer 16 of 32, last token**
Matrix Z (K × d)	3 × 4	**10 × 4,096**
Covariance C	3 × 3	**10 × 10**
Eigenvalues	3	**10**

7. Which Layer We Read, and Why



Layer 16 (middle): we read the vector of the **last token** of each answer: 4,096 numbers.

Why the middle? Early layers encode surface word patterns; the final layers focus on predicting the next word. The middle layers carry the most **meaning**.

Why the last token? The model reads left to right, so the last token has "seen" the whole answer.

Layer 31: where **feature clipping** caps each of its 4,096 neurons to its normal range (0.2–99.8 percentile of the last 3,000 tokens).

8. Engineering Challenges We Solved

Challenge	Our solution
7B model needs 13.5 GB; the GPU has 8 GB	4-bit NF4 weights: **3.9 GB**
Saving all 33 layers for every token ran out of memory on long stories	Hook keeps **only layer 16's last-token vector** (test: identical result)
10 long answers in one pass is memory-heavy	Generate **2 at a time × 5**, pool all 10 before scoring
Proving the maths is implemented correctly	**8 unit tests**, e.g. clip limits vs. numpy.percentile (8/8 pass)

9. Python Workflow

1. Load **Llama-2-7B** in 4-bit (`transformers` + `bitsandbytes` NF4): 13.5 GB → 3.9 GB — `32 layers`
2. `model.generate()` samples **10 answers** (temperature 0.5, top-p 0.99) — `K = 10`
3. **Forward hook on layer 16** stores the last-token vector of each answer — `Z: 10 × 4096`
4. `torch.cov(Z)`: covariance between the 10 answers — `C: 10 × 10`
5. `np.linalg.svd(C + 0.001·I)`: eigenvalues — `10 values`
6. `np.mean(np.log10(λ))`: hallucination score — `1 number`
+ **Feature clipping** (optional): hook on layer 31 clamps extreme activations during generation — `4096 neurons`
✓ **Evaluation:** `rouge_score` marks each answer right/wrong (ROUGE-L > 0.5); `sklearn` computes AUROC — `250 questions`

10. Results

The LLM's own answers on 250 test questions (CoQA)

92 correct (37%) | 158 wrong (63%)

83.5% Take one **wrong** and one **correct** answer at random: in **83.5% of cases** our score is higher for the wrong one. (Random guessing: 50%. Perfect: 100%. Metric name: AUROC.)

Result	Value
Detection score (AUROC)	**83.5%**
With feature clipping on	**82.5%**
Time per question (10 answers + score)	**≈ 6 s**
GPU memory for the model	**≈ 3.9 GB of 8 GB**

11. Conclusion & Limitations

Using only the model's internal states (layer 16, 10 answers), our system flags likely-wrong answers with an **83.5% detection score**, needs no answer key, and runs on an 8 GB laptop GPU.

- **Cost:** each question needs 10 generated answers.
- **Access:** needs the model's internal layers, so it works with open models (not closed chat APIs).
- **Scope:** it flags likely errors but does not correct them; tested on 250 CoQA questions.

12. References

[1] C. Chen et al., "INSIDE: LLMs' Internal States Retain the Power of Hallucination Detection," ICLR 2024. [2] S. Reddy et al., "CoQA," TACL 2019. [3] H. Touvron et al., "Llama 2," arXiv:2307.09288, 2023.